LAPORAN PRAKTIKUM
STRUKTUR DATA

Stack Array

disusun oleh :

Khairunnisa M.

NIM 2511532005

Dosen Pengampu :

Dr. Wahyudi,. S.T., M.T.

Asisten Dosen :

M. Ghalid

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji Syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa ata selesainya laporan praktikum Pekan 3 mata kuliah Struktur Data. Laporan ini disusun untuk mendokumentasikan hasil pemahaman praktukum mengenai struktur data Stack serta implementasinya dalam Bahasa pemrograman Java.

Penulis menyadari bahwa keberasilan praktikum ini tidak lepas dari bimbingan dosen dan asisten praktikum. Penulis berharap laporan ini dapat bermanfaat bagi pembaca dan membantu memperdalam pemahaman mengenai konsep LIFO dalam pengembangan aplikasi.

Padang, 18 April 2026

Khairunnisa M.

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
BAB I 1
PENDAHULUAN 1
1.1. Latar Belakang 1
1.2. Tujuan Praktikum 1
1.3. Manfaat Praktikum 1
BAB II 3
PEMBAHASAN 3
2.1. Deskripsi Materi StackArray 3
2.2. StackArray_2511532005 4
2.3. StackArrayDriver_2511532005. 5
2.4. StackPostfix_2511532005 6
2.5. NilaiMaksimum_2511532005 7
2.6. SiswaStack_2511532005 8
2.7. Analisis Praktikum 9
BAB III 11
KESIMPULAN DAN SARAN 11
3.1 Kesimpulan 11
3.2 Saran 11
DAFTAR PUSTAKA 12

BAB I

PENDAHULUAN

1.1. Latar Belakang

Dalam dunia pemrograman, efisiensi manajemen data sangat bergantung pada pemilihan struktur data yang tepat. Salah satu struktur data linear yang sangat penting adalah Stack. Berbeda dengan *ArrayList* yang memungkinkan akses acak, Stack bekerja dengan prinsip LIFO (Last In First Out), di mana elemen yang terakhir masuk akan menjadi yang pertama keluar. Konsep ini sangat krusial dalam berbagai fungsi sistem komputer, seperti fitur *undo/redo* pada editor teks, proses navigasi *back* pada browser, hingga evaluasi ekspresi matematika. Melalui praktikum ini, praktikan akan mempelajari bagaimana membangun struktur Stack baik secara manual menggunakan array maupun memanfaatkan *Java Collection Framework*.

1.2. Tujuan Praktikum

1. Memahami prinsip LIFO
2. Menguasai operasi dasar Stack
3. Implementasi Stack manual
4. Pemanfaatan Java collection Framework
5. Penerapan Logika Algoritma

1.3. Manfaat Praktikum

1. Melatih logika terstruktur dalam mengelola alur data yang bersifat sekuensial dan bertumpuk.
2. Optimasi manajemen memori melalui perbandingan implementasi stack.

3. Kesiapan pemecahan masalah kompleks
4. Keahlian pemrograman modular
5. Efisiensi penulisan kode program

BAB II

PEMBAHASAN

2.1. Deskripsi Materi StackArray

Stack (Tumpukan) adalah struktur data linear yang mengikuti prinsip LIFO (Last In First Out), di mana elemen yang terakhir dimasukkan ke dalam tumpukan akan menjadi elemen yang pertama kali dikeluarkan. Dalam implementasi StackArray, sebuah array statis digunakan sebagai wadah penyimpanan utama untuk elemen-elemen tersebut.

Dalam mekanisme ini, terdapat sebuah variabel penunjuk yang biasanya disebut sebagai top. Variabel ini berfungsi untuk menyimpan indeks dari elemen yang berada di posisi paling atas tumpukan. Berikut adalah detail operasi utama pada StackArray:

- Push: Menambahkan elemen baru ke atas tumpukan. Sebelum melakukan *push*, program harus memastikan bahwa tumpukan tidak penuh (*Stack Overflow*). Secara teknis, nilai top akan bertambah (top = top + 1) lalu data dimasukkan ke indeks tersebut.
- Pop: Mengambil dan menghapus elemen teratas dari tumpukan. Sebelum melakukan *pop*, program harus memastikan tumpukan tidak kosong (*Stack Underflow*). Setelah data diambil, nilai top akan berkurang (top = top - 1).
- Peek/Top: Melihat nilai elemen yang berada di posisi teratas tanpa menghapusnya dari tumpukan.
- isEmpty: Operasi logika untuk mengecek apakah tumpukan kosong (biasanya ditandai dengan (top = - 1).
- isFull: Operasi logika untuk mengecek apakah tumpukan sudah mencapai kapasitas maksimal array.

Karakteristik utama dari StackArray adalah ukurannya yang tetap (*fixed-size*). Karena menggunakan array sebagai basisnya, kapasitas

tumpukan harus ditentukan di awal program, sehingga penggunaan memori menjadi sangat terukur namun kurang fleksibel jika dibandingkan dengan implementasi berbasis *linked list* atau *ArrayList*.

2.2. StackArray_2511532005

Progam ini adalah implementasi Stack secara manual menggunakan array statsi sebagai media penyimpanan.

```java
package pekan3_2511532005;

public class stackArray_2511532005 {
    static final int MAX_2005 = 1000;
    int top_2005;
    int a_2005[] = new int[MAX_2005];
    boolean isEmpty()
    {
        return (top_2005 < 0);
    }
    stackArray_2511532005()
    {
        top_2005 = -1;
    }
    boolean push_2005 (int x)
    {
        if (top_2005 >= (MAX_2005 - 1)) {
            System.out.println("Stack Overflow");
            return false;
        }
        else {
            a_2005 [++ top_2005] = x;
            System.out.println(x + " dimasukkan dalam stack ");
            return true;
        }
    }
    int pop_2005()
    {
        if (top_2005 < 0) {
            System.out.println(" Stack Underflow");
            return 0;
        }
        else {
            int x = a_2005 [top_2005 --];
            return x;
        }
    }
    int peek_2005()
    {
        if (top_2005 < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a_2005[top_2005];
            return x;
        }
    }
    void print_2005() {
        for (int i = top_2005; i>-1;i--) {
            System.out.print (" " + a_2005[i]);
        }

    }

}
```

Gambar 2.1. Kode program StackArray_2511532005.

Kode dan Fungsi :

1. static final int MAX_2005 = 1000; : menetapkan kapasitas maksimum tumpukan.
2. int top_2005; : variable penanda indeks elemen teratas. Nilai -1 berarti kosong.
3. isEmpty(); : memeriksa apakah tumpukan kosong berdasarkan nilai top.
4. push_2005(int x); : memasukkan data ke tumpukan. Ada pengecekan agar tidak terjadi *Stack Overflow*.
5. pop_2005() : mengambil dan menghapus data teratas. Memiliki pengecekan agar tidak terjadi *Stack Underflow*.
6. peek_2005() : melihat nilai teratas tanpa menghapusnya.
7. print_2005() : Iterasi mundur dari top ke 0 untuk mencetak seluruh isi tumpukan seusai urutan keluarannya.

2.3. StackArrayDriver_2511532005.

Kelas ini berfungsi sebagai penguji untuk memastikan logika kelas stackArray berjalan dengan benar.

```java
package pekan3_2511532005;

public class stackArrayDriver_2511532005 {

    public static void main(String[] args) {
        stackArray_2511532005 s_2005 = new stackArray_2511532005();
        s_2005.push_2005 (10);
        s_2005.push_2005 (20);
        s_2005.push_2005(30);
        System.out.println(s_2005.pop_2005() + " dikeluarkan dari stack");
        System.out.println(" Elemen teratas adalah :" + s_2005.peek_2005());
        System.out.print("Element pada Stack :");
        s_2005.print_2005();
    }

}
```

Gambar 2.2. Kode program StackArrayDriver_2511532005.

Kode dan fungsi :

1. instansiasi objek s_2005.
2. Menggunakan push_2005 untuk memasukkan nilai 10, 20, dan 30.
3. Memanggil pop_2005 untuk membuktikan bahwa nilai 30 (terakhir masuk) adalah yang pertama keluar.
4. Mencetak sisa elemen dalam tumpukan.

2.4. StackPostfix_2511532005

Program ini menunjukkan aplikasi Stack yang lebih kompleks, yaitu mengevaluasi ekspresi matematika dalam format Postfix (missal : “ 5 2 4* + 7 – “).

```java
package pekan3_2511532005;
import java.util.Scanner;

public class StackPostfix_2511532005 {
    public static int postfixEvaluate (String expression) {
        Stack<Integer> s_2005 = new Stack<Integer>();
        Scanner input_2005 = new Scanner(expression);
        while (input_2005.hasNext()) {
            if (input_2005.hasNextInt()) { // an operand (integer)
                s_2005.push (input_2005.nextInt());
            } else {                        // an operator
                String operator_2005 = input_2005.next();
                int operand2_2005 = s_2005.pop ();
                int operand1_2005 = s_2005.pop ();
                if (operator_2005.equals("+")) {
                    s_2005.push (operand1_2005 + operand2_2005);
                } else if (operator_2005.equals ("-")) {
                    s_2005.push(operand1_2005 - operand2_2005);
                } else if (operator_2005.equals("*")) {
                    s_2005.push(operand1_2005 * operand2_2005);
                } else {
                    s_2005.push (operand1_2005 / operand2_2005);
                }

            }
        }
        input_2005.close();
        return s_2005.pop();
    }
    public static void main (String[] args) {
        System.out.println (" Hasil postfix = " + postfixEvaluate (" 5 2 4 * + 7 -"));
    }
}
```

Gambar 2.3. Kode Program StackPostfix_2511532005

Kode dan fungsi :

1. Stack<Integer> s_2005 : menggunakan kelas Stack bawaan Java.
2. Scanner input_2005 : membedah string ekspresi datu per satu.
3. Jika input berupa angka (hasNextInt), maka di-push ke tumpukan.

4. Jika berupa operator (+, -, *, /) program melakukan pop dua kali (mengambil dua angka terakhir), menghitung hasilnya, lalu men push Kembali hasilnya ke tumpukan.
5. Akhir program akan menghasilkan satu nilai tungga sebagai hasil evaluasi.

2.5. NilaiMaksimum_2511532005

Metode ini digunakan untuk mencari nilai terbesar di dalam sebuah tumpukan tanpa merusak urutan data aslinya

```java
package pekan3_2511532005;
import java.util.Stack;

public class NilaiMaksimum_2511532005 {
    public static int max_2005 (Stack<Integer> s_2005) {
        Stack<Integer> backup_2005 = new Stack<Integer> ();
        int maxValue_2005 = s_2005.pop();
        backup_2005.push (maxValue_2005);
        while (!s_2005.isEmpty()) {
            int next_2005 = s_2005.pop();
            backup_2005.push (next_2005);
            maxValue_2005 = Math.max(maxValue_2005,next_2005);
        }
        while (!backup_2005.isEmpty()) {
            s_2005.push (backup_2005.pop());
        }
        return maxValue_2005;
    }
    public static void main (String[] args) {
        Stack<Integer> s_2005 = new Stack<Integer> ();
        s_2005.push (70);
        s_2005.push (12);
        s_2005.push (20);
        System.out.println (" Isi stack " + s_2005);
        System.out.println (" Stack Teratas " + s_2005.peek());
        System.out.println ("Nilai maksimum " + max_2005(s_2005));
    }

}
```

Gambar 2.4. Kode program NilaiMaksimum_2511532005.

Kode dan fungsi :

1. Stack<Integer> backup_2005 : tumpukan sementara untuk menampung data yang sedang diperiksa.
2. maxValue_2005 = s_2005.pop() : inisialisasi nilai maksimum awal.

3. Perulangan while : mengambil semua data dari tumpukan asli, membandingkan dengan nilai maksimum saat ini menggunakan Math.max() , lalu memindahkan data ke backup.
4. Perulangan kedua : mengembalikan semua data dari backup ke tumpukan asli (s_2005) agar kondisi tumpukan tetap utuh seperti semula.

2.6. SiswaStack_2511532005

Implementasi Stack untuk mengelola objek buatan sendiri (*User-Defined Object*), yaitu data siswa.

```java
package pekan3_2511532005;
import java.util.ArrayList;

class Siswa_2005 {
    String nama_2005;
    int nim_2005;

    public Siswa_2005 (String nama_2005, int nim_2005) {
        this.nama_2005 = nama_2005;
        this.nim_2005 = nim_2005;
    }

    @Override
    public String toString() {
        return " NIM : " + nim_2005 + "NAMA : " + nama_2005;
    }
}
public class SiswaStack_2511532005 {
    private ArrayList<Siswa_2005> stack;

    public SiswaStack_2511532005() {
        stack = new ArrayList<>();
    }

    public void push_2005 (Siswa_2005 mhs) {
        stack.add(mhs);
    }
    public Siswa_2005 pop_2005() {
        if (!isEmpty_2005()) {
            return stack.remove(stack.size() - 1);
        }
        return null;
    }

    public Siswa_2005 peek_2005() {
        if (!isEmpty_2005()) {
            return stack.get(stack.size() - 1);
        }
        return null;
    }
```

```java

    public boolean isEmpty_2005() {
        return stack.isEmpty();
    }

    public void tampilkanSiswa_2005() {
        for (int i_2005 = stack.size() - 1 ; i_2005>= 0; i_2005--) {
            System.out.println(stack.get(i_2005));
        }
    }
    public static void main(String[] args) {
        SiswaStack_2511532005 studentStack= new SiswaStack_2511532005();

        Siswa_2005 mhs1 = new Siswa_2005 ("Ali", 1);
        Siswa_2005 mhs2 = new Siswa_2005 ("Bobby", 2);
        Siswa_2005 mhs3 = new Siswa_2005 ("Charles", 3);
        studentStack.push_2005(mhs1);
        studentStack.push_2005(mhs2);
        studentStack.push_2005(mhs3);

        System.out.println ("Siswa di dalam stack:");
        studentStack.tampilkanSiswa_2005();

        System.out.println("Siswa teratas " + studentStack.peek_2005());
        System.out.println(" mengeluarkan siswa teratas dari stack :" + studentStack.pop_2005());
        System.out.println(" daftar siswa setelah di pop :");
        studentStack.tampilkanSiswa_2005();
    }
}

```

Gambar 2.5. Kode program SiswaStack_2511532005.

Kode dan fungsi :

1. class Siswa_2005 : model data yang menyimpan nama dan nim.
2. ArrayList<Siswa_2005> stack: menggunakan ArrayList sebagai dasar penyimpanan tumpukan.
3. push_2005(Siswa_2005 mhs) : menambahkan objek siswa ke akhir list (posisi atas).
4. pop_2005() : menghapus data pada indeks size() -1 untuk mensimulasikan pengambilan dari atas.
5. tampilkanSiswa_2005() : mencetak daftar siswa dengan urutan LIFO.

2.7. Analisis Praktikum

Berdasarkan hasil praktikum, dapat dianalisis bahwa struktur data Stack sangat bergantung pada variabel penunjuk atau *top index* untuk mengelola alur data. Dalam implementasi StackArray, penggunaan array statis mengharuskan adanya validasi ketat terhadap kapasitas maksimal guna menghindari *Stack Overflow*, serta pengecekan kondisi kosong untuk mencegah *Stack Underflow* saat operasi pop atau peek dilakukan. Prinsip LIFO (*Last In First Out*) teramati

dengan jelas pada kelas driver, di mana angka terakhir yang dimasukkan (30) menjadi elemen pertama yang keluar, membuktikan bahwa Stack hanya memberikan akses pada elemen paling atas (top). Hal ini berbeda dengan *ArrayList* yang memungkinkan akses acak, karena Stack lebih fokus pada urutan proses yang bersifat sekuensial terbalik.

Analisis lebih lanjut pada program StackPostfix dan NilaiMaksimum menunjukkan fleksibilitas Stack dalam menyelesaikan masalah logika yang kompleks. Pada evaluasi postfix, Stack berperan sebagai tempat penyimpanan sementara bagi operan hingga ditemukan operator aritmatika, yang memungkinkan penghitungan matematika tanpa memerlukan tanda kurung. Sementara itu, pada kasus NilaiMaksimum dan SiswaStack, terlihat bahwa Stack dapat digunakan untuk menyimpan objek kustom dan dilakukan pemrosesan nilai menggunakan bantuan tumpukan sementara (*backup stack*). Penggunaan tumpukan cadangan ini sangat penting dalam analisis data Stack, karena memungkinkan kita untuk memeriksa atau mencari data di lapisan bawah tumpukan tanpa merusak atau menghilangkan urutan data aslinya.

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Melalui seluruh rangkaian praktikum di pekan ini, dapat disimpulkan bahwa Stack merupakan struktur data yang sangat krusial dalam pemrosesan data dengan alur terbalik (LIFO). Implementasi Stack dapat dilakukan secara statis menggunakan array biasa maupun secara dinamis menggunakan ArrayList atau kelas Stack bawaan Java. Praktikan juga berhasil memahami bahwa Stack sangat efektif digunakan untuk operasi pembedahan ekspresi matematika (*parsing*) dan manajemen tumpukan objek kompleks. Keamanan data dalam Stack terjaga dengan baik melalui pengecekan kondisi *Overflow* dan *Underflow*, sehingga mencegah error saat tumpukan penuh atau kosong.

3.2 Saran

Meskipun implementasi dasar telah berhasil dilakukan, disarankan untuk mulai mengeksplorasi penggunaan Stack dalam kasus navigasi aplikasi (seperti tombol "kembali"). Selain itu, penggunaan Exception Handling (seperti try-catch) akan jauh lebih baik daripada sekadar mencetak pesan error pada konsol saat terjadi tumpukan penuh atau kosong. Praktikan juga disarankan untuk mempelajari struktur data *Queue* (Antrian) di pertemuan berikutnya agar dapat membandingkan prinsip LIFO dengan prinsip FIFO secara lebih mendalam.

DAFTAR PUSTAKA

Juliansyah, N., Sari, S. Y., & Dristyan, F. (n.d.). *Optimasi Struktur Data Stack dan Queue Menggunakan Array Dinamis.* Retrieved from ejurnal.faaslibsmedia: https://ejurnal.faaslibsmedia.com/index.php/fusion/article/view/264

Selamet, R. (2016). *IMPLEMENTASI STRUKTUR DATA LIST, QUEUE DAN STACK DALAM JAVA .* Retrieved from Academia: https://d1wqtxts1xzle7.cloudfront.net/87054128/112016_03_Rachmat-libre.pdf?1654492616=&response-content-disposition=inline%3B+filename%3DImplementasi_Struktur_Data_List_Queue_Da.pdf&Expires=1776614990&Signature=LZ0TcA2YJyjyHltodY96UmhWcfhaWOxJuEKFJyDarN-6

trivusi. (2022). *Struktur Data Stack: Pengertian, Karakteristik, dan Kegunaannya*. Retrieved from trivusi: https://www.trivusi.web.id/2022/07/struktur-data-stack.html

TUGAS PRAKTIKUM 3 StackArray

1. Perintah Tugas

Pada pekan ini, mahasiswa diminta untuk mengimplementasikan konsep Stack (Tumpukan) yang bekerja dengan prinsip LIFO (Last In, First Out). Tema tugas kali ini adalah Simulasi Browser History. Mahasiswa diwajibkan membuat program yang mensimulasikan perpindahan halaman web. Halaman yang terakhir dibuka akan berada di posisi teratas tumpukan, dan menjadi halaman pertama yang dihapus saat tombol "Back" ditekan.

Gambar 1.1 Perintah Tugas Praktikum Pekan 3 .

2. Kode Program Website_2511532005

Berikut Kode Program Website_2511532005.

```java
package pekan3_2511532005;

public class Website_2511532005 {
    // Atribut dengan akhiran 4 digit terakhir NIM (2005)
    private String judul_2005;
    private String url_2005;

    // Constructor
    public Website_2511532005(String judul, String url) {
        this.judul_2005 = judul;
        this.url_2005 = url;
    }

    // Getter (Selektor)
    public String getJudul_2005() {
        return judul_2005;
    }

    public String getUrl_2005() {
        return url_2005;
    }

    // Setter (Mutator)
    public void setJudul_2005(String judul) {
        this.judul_2005 = judul;
    }

    public void setUrl_2005(String url) {
        this.url_2005 = url;
    }

    // Method untuk menampilkan detail website
    public void tampilkanWebsite() {
        System.out.println("Judul: " + judul_2005);
        System.out.println("URL: " + url_2005);
        System.out.println("-----------------------");
    }
}
```

Gambar 2.1 Kode Program Mobil_2511532005.

Penjelasan fungsi atau kegunaan dari kode program ini :

a. Deklarasi Class : sebagai template untuk membuat objek website.

b. Atribut
   - private String judul_2005; menyimpan nama/judul website.
   - private String url_2005; menyimpan alamat website.

c. Constructor : digunakan untuk inisialisasi saat pertama kali dibuat

d. Getter ; untuk mengambil nilai atribut dari luar kelas.

e. Setter ; mengubah nilai atribut dari luar kelas.

f. Method tampilkanWebsite(); menampilkan informasi website ke layar.

3. Kode Program Browser_2511532005

```java
package pekan3_2511532005;
import java.util.Stack;
import java.util.Scanner;

public class Browser_2511532005 {
    private Stack<Website_2511532005> history;  // Stack LIFO
    private Scanner input;

    public Browser_2511532005() {
        history = new Stack<>();
        input = new Scanner(System.in);
    }

    public void tampilkanMenu() {
        System.out.println("=== Browser History NIM: 2511532005 ===");
        System.out.println("1. Kunjungi Website (Push)");
        System.out.println("2. Tombol Back (Pop)");
        System.out.println("3. Lihat Halaman Aktif (Peek)");
        System.out.println("4. Cek Status History");
        System.out.println("5. Keluar");
        System.out.print("Pilihan: ");
    }

    // 1. PUSH - Kunjungi Website Baru
    public void kunjungiWebsite() {
        System.out.print("Masukkan Judul: ");
        String judul = input.nextLine();

        System.out.print("Masukkan URL: ");
        String url = input.nextLine();

        Website_2511532005 websiteBaru = new Website_2511532005(judul, url);
        history.push(websiteBaru);  // ☑ PUSH ke Stack (LIFO)

        System.out.println("\nBerhasil mengunjungi halaman!");
        System.out.println();
    }

```

```java
    // 2. POP - Tombol Back
    public void tombolBack() {
        if (history.isEmpty()) {  // ☑ Handle EmptyStackException
            System.out.println("History kosong! Tidak ada halaman untuk kembali.");
        } else {
            Website_2511532005 halamanTerakhir = history.pop();  // ☑ POP (LIFO)
            System.out.println("Kembali ke halaman sebelumnya:");
            System.out.println("Halaman yang ditutup: " + halamanTerakhir.getJudul_2005());
        }
        System.out.println();
    }

    // 3. PEEK - Lihat Halaman Aktif
    public void lihatHalamanAktif() {
        if (history.isEmpty()) {
            System.out.println("History kosong! Belum ada halaman yang dikunjungi.");
        } else {
            Website_2511532005 halamanAktif = history.peek();  // ☑ PEEK (tanpa hapus)
            System.out.println("Halaman Aktif Saat Ini:");
            halamanAktif.tampilkanWebsite();
        }
        System.out.println();
    }

    // 4. CEK STATUS HISTORY
    public void cekStatusHistory() {
        if (history.isEmpty()) {
            System.out.println("History kosong. Belum ada riwayat penjelajahan.");
        } else {
            System.out.println("Jumlah total riwayat: " + history.size());
            System.out.println("Halaman teratas: " + history.peek().getJudul_2005());
        }
        System.out.println();
    }

    public void run() {
        int pilihan;
        do {
            tampilkanMenu();
            pilihan = input.nextInt();
            input.nextLine(); // Clear buffer

            switch (pilihan) {
                case 1:
                    kunjungiWebsite();
                    break;
                case 2:
                    tombolBack();
                    break;
                case 3:
                    lihatHalamanAktif();
                    break;
                case 4:
                    cekStatusHistory();
                    break;
                case 5:
                    System.out.println("Terima kasih!");
                    break;
                default:
                    System.out.println("Pilihan tidak valid!");
                    System.out.println();
            }
        } while (pilihan != 5);
    }

    public static void main(String[] args) {
        Browser_2511532005 browser = new Browser_2511532005();
        browser.run();
    }
}
```

Gambar 3.1 Kode Program Browser_2511532005.

Penjelasan fungsi atau kegunaan dari kode program ini :

a. Atribut.

- history untuk tempat menyimpan objek website_2511532005.
- Input Adalah alat untuk membaca input dari keyboard.

b. Constructor, membuat stack kosong dan mengaktifkan scanner.

c. Method tampilkanMenu(); menampilkan menu pilihan ke user.

d. Method kunjungiWebsite(); menambahkan halaman bar uke history.
e. Method tombolBack(); menghapus halaman terakhir.
f. Method lihatHalamanAktif(); melihat halaman yang sedang aktif tanpa menghapusnya.
g. Method cekStatusHistory(); untuk mengetahui kondisi stack.
h. Method run(); loop utama program.
i. Method main(); titik awal program.

4. Output Dari Kode Program

```
=== Browser History NIM: 2511532005 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: spotify
Masukkan URL: www.spotify.com

Berhasil mengunjungi halaman!
```

```
=== Browser History NIM: 2511532005 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 2
Kembali ke halaman sebelumnya:
Halaman yang ditutup: spotify
```

```
=== Browser History NIM: 2511532005 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 3
History kosong! Belum ada halaman yang dikunjungi.
```

```
=== Browser History NIM: 2511532005 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 4
History kosong. Belum ada riwayat penjelajahan.
```

```
=== Browser History NIM: 2511532005 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 5
Terima kasih!
```